Sesión 5: MANIPULACIÓN DE DATOS Y LÓGICA DE CONJUNTOS

DML, JOINs, Subconsultas, CTEs y Optimización en SQL

Por

Carlos César Sánchez Coronel

2026

Índice general

Índice general **3**

1. Introducción a la Manipulación de Datos **4**
1.1. El Lenguaje SQL y sus Sublenguajes 4

2. Lenguaje SQL: Sublenguajes y Operaciones Fundamentales **5**
2.1. DDL (Data Definition Language) 5
2.1.1. CREATE 5
2.1.2. ALTER 5
2.1.3. DROP 6
2.1.4. TRUNCATE 6
2.1.5. Restricciones de Integridad en DDL 6
2.2. DML (Data Manipulation Language) 7
2.2.1. SELECT 7
2.2.2. INSERT 7
2.2.3. UPDATE 7
2.2.4. DELETE 7
2.2.5. MERGE (UPSERT) 8
2.3. DCL (Data Control Language) 8
2.3.1. GRANT 8
2.3.2. REVOKE 8
2.3.3. Casos de Uso en Entrevistas 8
2.4. TCL (Transaction Control Language) 8
2.4.1. BEGIN / START TRANSACTION 9
2.4.2. COMMIT 9
2.4.3. ROLLBACK 9
2.4.4. SAVEPOINT 9
2.4.5. Propiedades ACID 9
2.5. Comparativa entre Sublenguajes 9
2.6. Ejercicios Resueltos Integradores 10
2.6.1. Ejercicio 1: Creación y manipulación 10
2.6.2. Ejercicio 2: Transacción con error 10
2.6.3. Ejercicio 3: Permisos y seguridad 11
2.7. Conclusión 11

3. El Proceso Interno de una Consulta SQL **12**
3.1. Orden Lógico de Ejecución de una Consulta 12
3.1.1. Ejemplo 12

3.2. El Camino Físico: De la Consulta al Hardware . . . 13
3.2.1. Parsing (Análisis Sintáctico) . . . 13
3.2.2. Optimización . . . 13
3.2.3. Ejecución . . . 13
3.2.4. Retorno de Resultados . . . 13
3.2.5. Hardware Involucrado . . . 13
3.3. EXPLAIN: La Herramienta Clave . . . 14

4. Joins: Combinando Tablas 15
4.1. Producto Cartesiano y JOIN . . . 15
4.2. Tipos de JOIN . . . 15
4.2.1. INNER JOIN . . . 15
4.2.2. LEFT JOIN (o LEFT OUTER JOIN) . . . 16
4.2.3. RIGHT JOIN . . . 16
4.2.4. FULL OUTER JOIN . . . 16
4.2.5. CROSS JOIN . . . 16
4.2.6. Self-Join . . . 17
4.2.7. JOIN con condiciones no equitativas . . . 17
4.3. Algoritmos de JOIN Internos . . . 17
4.4. Ejemplo Práctico con Múltiples Tablas . . . 17

5. Subconsultas 19
5.1. Subconsultas Escalares . . . 19
5.2. Subconsultas de Fila Única . . . 19
5.3. Subconsultas de Múltiples Filas . . . 19
5.4. Subconsultas Correlacionadas . . . 20
5.5. Subconsultas en la cláusula FROM . . . 20

6. Common Table Expressions (CTEs) 21
6.1. Sintaxis Básica . . . 21
6.1.1. Ejemplo . . . 21
6.1.2. CTEs Múltiples . . . 21
6.1.3. CTEs Recursivas . . . 22
6.2. CTEs vs. Subconsultas vs. Tablas Temporales . . . 22

7. Buenas Prácticas y Optimización de Consultas SQL 23
7.1. Preferir EXISTS a IN cuando la subconsulta es grande . . . 23
7.2. Evitar Funciones en Columnas Indexadas . . . 23
7.3. Usar UNION ALL en lugar de UNION si no se necesitan duplicados . . . 23
7.4. SELECT * Es Peligroso . . . 24
7.5. Cuidado con las Conversiones Implícitas . . . 24
7.6. Índices y JOINs . . . 24
7.7. Analizar el Plan con EXPLAIN . . . 24
7.8. Escritura de Consultas Legibles . . . 24

8. Consideraciones de Hardware y Motor 25
8.1. Buffer Pool y Caché . . . 25
8.2. Disco: SSD vs HDD . . . 25
8.3. Parallel Query . . . 25

8.4. Configuración del Motor 25

9. Breve Comparación entre Motores (para JOINs y DML) 26

10. Joins en el Mundo NoSQL (Una Breve Mirada) 27

11. Ejercicios Resueltos 28

11.1. Ejercicio 1: Consultas Básicas con DML 28
11.2. Ejercicio 2: INNER JOIN y LEFT JOIN 28
11.3. Ejercicio 3: Subconsultas 29
11.4. Ejercicio 4: Uso de CTE 29
11.5. Ejercicio 5: Optimización y EXPLAIN 30
11.6. Ejercicio 6: Recursividad con CTE 30

12. Glosario de Términos 31

Referencias 32

Capítulo 1

Introducción a la Manipulación de Datos

En las sesiones anteriores hemos aprendido a modelar bases de datos, normalizar y definir estructuras. Ahora es el momento de interactuar con los datos: insertarlos, consultarlos, modificarlos y eliminarlos. El lenguaje SQL (Structured Query Language) es el estándar para estas tareas. En esta sesión nos centraremos en el sublenguaje DML (Data Manipulation Language) y en las operaciones de conjuntos que nos permiten combinar información de múltiples tablas: los JOINs. También exploraremos cómo el motor de base de datos interpreta y ejecuta nuestras consultas, qué sucede a nivel hardware, y cómo escribir SQL eficiente. Dominar estos conceptos es fundamental para cualquier profesional que trabaje con datos, ya que constituyen la base de cualquier análisis, informe o aplicación, y son temas recurrentes en entrevistas técnicas nacionales e internacionales.

1.1. El Lenguaje SQL y sus Sublenguajes

SQL se divide en varios sublenguajes según su función:

- **DDL (Data Definition Language):** CREATE, ALTER, DROP – definen la estructura.
- **DML (Data Manipulation Language):** SELECT, INSERT, UPDATE, DELETE – manipulan los datos.
- **DCL (Data Control Language):** GRANT, REVOKE – controlan permisos.
- **TCL (Transaction Control Language):** COMMIT, ROLLBACK – gestionan transacciones.

En esta sesión profundizaremos en DML, con especial énfasis en SELECT y JOINs.

Capítulo 2

Lenguaje SQL: Sublenguajes y Operaciones Fundamentales

El lenguaje SQL se divide en varios sublenguajes según la naturaleza de las operaciones. Conocer cada uno y sus comandos es esencial para cualquier profesional de bases de datos, ya que permite definir estructuras, manipular datos, controlar permisos y gestionar transacciones. En este capítulo exploraremos los cuatro sublenguajes principales: DDL, DML, DCL y TCL, con ejemplos prácticos y casos de uso que suelen aparecer en entrevistas técnicas.

2.1. DDL (Data Definition Language)

El sublenguaje de definición de datos se utiliza para crear, modificar y eliminar la estructura de los objetos de la base de datos (tablas, índices, vistas, etc.). Los comandos DDL suelen ser auto-commit, es decir, no se pueden deshacer con ROLLBACK en la mayoría de los motores.

2.1.1. CREATE

Crea nuevos objetos. El más común es `CREATE TABLE`.

```
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    ciudad VARCHAR(50),
    fecha_registro DATE DEFAULT CURRENT_DATE
);
```

También se pueden crear índices, vistas, etc.:

```
CREATE INDEX idx_ciudad ON clientes(ciudad);
CREATE VIEW clientes_madrid AS SELECT * FROM clientes WHERE
   ciudad = 'Madrid';
```

2.1.2. ALTER

Modifica la estructura de un objeto existente.

```
-- A adir una columna
ALTER TABLE clientes ADD COLUMN telefono VARCHAR(20);

-- Modificar tipo de dato
ALTER TABLE clientes ALTER COLUMN nombre TYPE VARCHAR(200);

-- A adir una restricci n
ALTER TABLE clientes ADD CONSTRAINT email_unico UNIQUE (email);
```

2.1.3. DROP

Elimina objetos de la base de datos. ¡Cuidado! Es irreversible (a menos que tengas backup).

```
DROP TABLE clientes; -- Elimina la tabla y todos sus datos
DROP INDEX idx_ciudad;
DROP VIEW clientes_madrid;
```

2.1.4. TRUNCATE

Elimina todas las filas de una tabla de forma rápida (no genera logs por fila, a diferencia de DELETE). Es DDL porque no se puede deshacer con ROLLBACK en muchos motores.

```
TRUNCATE TABLE clientes;
```

2.1.5. Restricciones de Integridad en DDL

Las restricciones (constraints) se definen en DDL y garantizan la integridad de los datos:

- **PRIMARY KEY**: Identifica unívocamente cada fila.
- **FOREIGN KEY**: Mantiene integridad referencial.
- **UNIQUE**: Valores únicos en una columna o combinación.
- **CHECK**: Valida que los datos cumplan una condición (ej. salario ¿0).
- **NOT NULL**: La columna no puede tener valores nulos.

Ejemplo con todas ellas:

```
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    cliente_id INT NOT NULL,
    fecha DATE NOT NULL,
    total DECIMAL(10,2) CHECK (total >= 0),
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);
```

2.2. DML (Data Manipulation Language)

El sublenguaje de manipulación de datos permite consultar, insertar, modificar y eliminar los datos almacenados. Es el más utilizado en el día a día.

2.2.1. SELECT

La sentencia SELECT recupera datos de una o más tablas. Ya la hemos visto en detalle, pero recordemos su sintaxis básica:

```
SELECT columnas
FROM tabla
WHERE condicion
ORDER BY columna
LIMIT n;
```

Ejemplo:

```
SELECT nombre, email FROM clientes WHERE ciudad = 'Barcelona'
   ORDER BY nombre;
```

2.2.2. INSERT

Inserta nuevas filas en una tabla.

```
INSERT INTO clientes (nombre, email, ciudad) VALUES
('Carlos Ruiz', 'carlos@mail.com', 'Valencia');
```

También se puede insertar desde una consulta:

```
INSERT INTO clientes_vip (nombre, email)
SELECT nombre, email FROM clientes WHERE ciudad = 'Madrid';
```

2.2.3. UPDATE

Modifica filas existentes.

```
UPDATE clientes SET ciudad = 'Sevilla' WHERE id = 5;
```

Siempre usar WHERE para no actualizar toda la tabla.

2.2.4. DELETE

Elimina filas.

```
DELETE FROM clientes WHERE email IS NULL;
```

Sin WHERE, elimina todas las filas (pero la tabla sigue existiendo). Para vaciado masivo, TRUNCATE es más rápido.

2.2.5. MERGE (UPSERT)

Algunos motores (SQL Server, Oracle, PostgreSQL con ON CONFLICT) permiten insertar o actualizar según exista o no la fila. Es muy útil en sincronización de datos.

```
-- PostgreSQL
INSERT INTO clientes (id, nombre, email)
VALUES (1, 'Ana', 'ana@mail.com')
ON CONFLICT (id) DO UPDATE SET nombre = EXCLUDED.nombre, email =
   EXCLUDED.email;
```

2.3. DCL (Data Control Language)

El sublenguaje de control de datos gestiona los permisos y accesos a los objetos de la base de datos. Es fundamental en entornos multiusuario y para cumplir normativas de seguridad.

2.3.1. GRANT

Otorga privilegios a usuarios o roles.

```
-- Otorgar permiso de SELECT sobre la tabla clientes al usuario '
   analista'
GRANT SELECT ON clientes TO analista;

-- Otorgar todos los privilegios sobre la base de datos
GRANT ALL PRIVILEGES ON DATABASE mi_bd TO admin;
```

Los privilegios comunes son: SELECT, INSERT, UPDATE, DELETE, REFERENCES, TRIGGER, CREATE, CONNECT, etc.

2.3.2. REVOKE

Revoca permisos previamente otorgados.

```
REVOKE DELETE ON clientes FROM analista;
```

2.3.3. Casos de Uso en Entrevistas

En entrevistas suelen preguntar cómo se gestionan los permisos en entornos multi-rol, o cómo asegurar que un usuario solo pueda leer datos. También es común preguntar sobre el principio de mínimo privilegio.

2.4. TCL (Transaction Control Language)

El sublenguaje de control de transacciones permite gestionar los cambios realizados con DML, agrupándolos en unidades lógicas que pueden confirmarse o deshacerse.

2.4.1. BEGIN / START TRANSACTION

Inicia una transacción. En algunos motores (PostgreSQL) se usa BEGIN; en MySQL, START TRANSACTION.

```
BEGIN;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
-- A n no son permanentes
```

2.4.2. COMMIT

Confirma todos los cambios realizados en la transacción actual, haciéndolos permanentes.

```
COMMIT;
```

2.4.3. ROLLBACK

Deshace todos los cambios realizados desde el BEGIN, volviendo al estado anterior.

```
ROLLBACK;
```

2.4.4. SAVEPOINT

Permite establecer puntos intermedios dentro de una transacción para poder deshacer solo una parte.

```
BEGIN;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
SAVEPOINT despues_de_restar;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
-- Si algo falla, podemos retroceder al savepoint
ROLLBACK TO SAVEPOINT despues_de_restar;
-- Luego podemos continuar o hacer COMMIT
COMMIT;
```

2.4.5. Propiedades ACID

Las transacciones garantizan las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), tema recurrente en entrevistas. Es importante saber explicarlas con ejemplos.

2.5. Comparativa entre Sublenguajes

A modo de resumen, la siguiente tabla muestra los comandos principales de cada sublenguaje:

Sublenguaje	Comandos principales	Uso
DDL	CREATE, ALTER, DROP, TRUNCATE	Definir estructura
DML	SELECT, INSERT, UPDATE, DELETE, MERGE	Manipular datos
DCL	GRANT, REVOKE	Controlar permisos
TCL	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	Gestionar transacciones

2.6. Ejercicios Resueltos Integradores

A continuación, ejercicios que combinan varios sublenguajes.

2.6.1. Ejercicio 1: Creación y manipulación

Enunciado: Crear una tabla `productos` con columnas id, nombre, precio, stock. Insertar tres productos. Actualizar el precio de uno. Eliminar un producto. Finalmente, otorgar permisos de SELECT a un usuario.

Solución:

```sql
-- DDL
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) CHECK (precio > 0),
    stock INT DEFAULT 0
);

-- DML
INSERT INTO productos (nombre, precio, stock) VALUES
('Laptop', 1200.00, 10),
('Mouse', 25.50, 50),
('Teclado', 45.00, 30);

UPDATE productos SET precio = 1150.00 WHERE nombre = 'Laptop';

DELETE FROM productos WHERE nombre = 'Mouse';

-- DCL
GRANT SELECT ON productos TO analista;
```

2.6.2. Ejercicio 2: Transacción con error

Enunciado: Realizar una transferencia bancaria entre dos cuentas, asegurando que si una operación falla, ninguna se aplique. Usar transacciones.

Solución:

```sql
BEGIN;
UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 500 WHERE id = 2;
-- Si todo va bien
COMMIT;
```

```
-- Si ocurre un error antes del COMMIT, se puede hacer ROLLBACK
```

2.6.3. Ejercicio 3: Permisos y seguridad

Enunciado: Crear un usuario 'lector' que solo pueda hacer SELECT en la tabla `clientes`, y un usuario 'editor' que pueda INSERT, UPDATE y DELETE. Implementar los comandos DCL.

Solución:

```
-- Crear usuarios (en PostgreSQL)
CREATE USER lector WITH PASSWORD 'pass1';
CREATE USER editor WITH PASSWORD 'pass2';

-- Otorgar permisos
GRANT SELECT ON clientes TO lector;
GRANT SELECT, INSERT, UPDATE, DELETE ON clientes TO editor;

-- Revocar permisos si es necesario
REVOKE DELETE ON clientes FROM editor;
```

2.7. Conclusión

Dominar los cuatro sublenguajes de SQL es fundamental para cualquier profesional de datos. No solo se trata de escribir consultas, sino de entender cómo definir estructuras robustas (DDL), manipular datos eficientemente (DML), controlar accesos (DCL) y garantizar la integridad transaccional (TCL). Estos conceptos son evaluados constantemente en entrevistas técnicas y son la base para construir sistemas de bases de datos seguros, escalables y confiables.

Capítulo 3

El Proceso Interno de una Consulta SQL

Para escribir SQL eficiente, es crucial entender cómo el motor procesa una consulta. No basta con saber la sintaxis; hay que comprender el flujo lógico y físico.

3.1. Orden Lógico de Ejecución de una Consulta

Aunque escribimos SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT, el motor ejecuta en un orden diferente. El orden lógico (conceptual) es:

1. **FROM y JOINs**: Se construye el conjunto de datos base (producto cartesiano de las tablas involucradas, aplicando las condiciones de JOIN).
2. **WHERE**: Se filtran las filas según las condiciones.
3. **GROUP BY**: Se agrupan las filas.
4. **HAVING**: Se filtran los grupos.
5. **SELECT**: Se evalúan las expresiones de las columnas, incluyendo funciones de agregación.
6. **ORDER BY**: Se ordena el resultado.
7. **LIMIT / OFFSET**: Se recorta el resultado.

Este orden es importante porque, por ejemplo, no se puede usar un alias de SELECT en WHERE (porque WHERE se ejecuta antes). Tampoco se pueden usar funciones de ventana en WHERE por la misma razón.

3.1.1. Ejemplo

```sql
SELECT nombre, salario, salario * 12 AS salario_anual
FROM empleados
WHERE salario > 3000
ORDER BY salario_anual;
```

Aquí, el alias `salario_anual` se puede usar en ORDER BY porque se calcula antes de ordenar, pero no se podría usar en WHERE.

3.2. El Camino Físico: De la Consulta al Hardware

Cuando ejecutamos una consulta, ocurren múltiples etapas:

3.2.1. Parsing (Análisis Sintáctico)

El motor verifica que la consulta sea sintácticamente correcta y que las tablas y columnas existan. Genera un árbol de consulta.

3.2.2. Optimización

El optimizador (basado en costos, CBO) genera varios planes de ejecución posibles y estima su costo (en términos de E/S, CPU, memoria). Considera:

- Estadísticas de las tablas (número de filas, distribución de valores, histogramas).
- Existencia de índices.
- Métodos de acceso (seq scan, index scan, bitmap scan).
- Algoritmos de JOIN (nested loop, hash join, merge join).

Elige el plan de menor costo estimado.

3.2.3. Ejecución

El plan se ejecuta, interactuando con el buffer cache y el almacenamiento. Si los datos están en caché (buffer pool), se evita E/S a disco. De lo contrario, se leen páginas del disco.

3.2.4. Retorno de Resultados

Las filas resultantes se envían al cliente, posiblemente en lotes.

3.2.5. Hardware Involucrado

- **CPU**: Procesa las condiciones, cálculos, agregaciones.
- **RAM (Buffer Pool)**: Almacena páginas de datos e índices recientemente usados.
- **Disco (SSD/HDD)**: Si hay cache miss, se lee de disco.
- **Red**: Transmite los resultados al cliente.

En entrevistas, preguntan cómo optimizar: reducir E/S, usar índices, evitar ordenamientos innecesarios, etc.

3.3. EXPLAIN: La Herramienta Clave

Para entender el plan de ejecución, usamos `EXPLAIN` (y `EXPLAIN ANALYZE` para ejecutar realmente y ver tiempos).

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM clientes WHERE ciudad =
    'Madrid';
```

Muestra el plan, costos estimados, filas reales, tiempo, y accesos a buffers (caché vs disco). Es esencial en entrevistas para demostrar capacidad de diagnóstico.

Capítulo 4

Joins: Combinando Tablas

Las bases de datos relacionales se basan en la normalización, que distribuye la información en múltiples tablas relacionadas mediante claves. Para recuperar datos que están dispersos, necesitamos combinar tablas mediante JOINs. Un JOIN combina filas de dos o más tablas basándose en una condición relacionada.

4.1. Producto Cartesiano y JOIN

Si escribimos:

```
SELECT * FROM A, B;
```

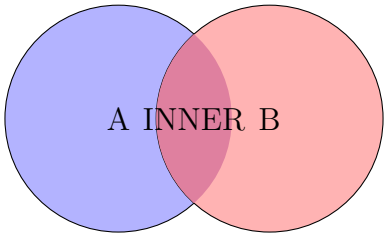
Obtenemos el producto cartesiano (todas las combinaciones de filas de A con B). Esto rara vez es útil. Los JOINs añaden una condición para filtrar las combinaciones relevantes.

4.2. Tipos de JOIN

Existen varios tipos, que se ilustran con diagramas de Venn (aunque hay que tener cuidado porque los conjuntos no son exactamente así, son ilustrativos).

4.2.1. INNER JOIN

Devuelve solo las filas que tienen correspondencia en ambas tablas.



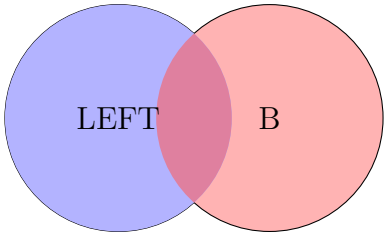
Sintaxis:

```
SELECT columnas
FROM tablaA
INNER JOIN tablaB ON tablaA.clave = tablaB.clave;
```

También se puede usar la palabra clave `JOIN` a secas (por defecto es INNER).

4.2.2. LEFT JOIN (o LEFT OUTER JOIN)

Devuelve todas las filas de la tabla izquierda, y las coincidentes de la derecha. Si no hay coincidencia, las columnas de la derecha son NULL.



Sintaxis:

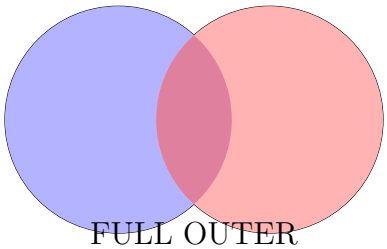
```
SELECT columnas
FROM tablaA
LEFT JOIN tablaB ON tablaA.clave = tablaB.clave;
```

4.2.3. RIGHT JOIN

Análogo al LEFT, pero con la tabla derecha como maestra. Es menos usado porque se puede escribir el LEFT cambiando el orden.

4.2.4. FULL OUTER JOIN

Devuelve todas las filas de ambas tablas, rellenando con NULL donde no hay coincidencia.



Sintaxis:

```
SELECT columnas
FROM tablaA
FULL OUTER JOIN tablaB ON tablaA.clave = tablaB.clave;
```

4.2.5. CROSS JOIN

Producto cartesiano (cada fila de A con cada fila de B). No lleva condición.

```
SELECT * FROM tablaA CROSS JOIN tablaB;
-- Equivalente a SELECT * FROM tablaA, tablaB;
```

4.2.6. Self-Join

Una tabla se une consigo misma. Es útil para relaciones jerárquicas (ej. empleados con jefe).

```
SELECT e1.nombre AS empleado, e2.nombre AS jefe
FROM empleados e1
LEFT JOIN empleados e2 ON e1.jefe_id = e2.id;
```

4.2.7. JOIN con condiciones no equitativas

La condición puede incluir otros operadores (¿, ¡, BETWEEN). Pero hay que tener en cuenta que estos joins pueden ser costosos.

```
SELECT *
FROM ventas v
JOIN productos p ON v.producto_id = p.id AND v.cantidad > p.
   stock_minimo;
```

4.3. Algoritmos de JOIN Internos

El motor elige entre varios algoritmos según el tamaño de las tablas y los índices:

- **Nested Loop:** Para cada fila de la tabla externa, busca coincidencias en la interna (ideal si una es pequeña o hay índice).
- **Hash Join:** Construye una tabla hash en memoria con una de las tablas, luego recorre la otra. Eficiente para tablas grandes sin índices.
- **Merge Join:** Ordena ambas tablas (si no lo están) y luego las fusiona. Útil si ya están ordenadas o se necesita ordenar de todas formas.

Saber cuándo se usa cada uno es una pregunta clásica en entrevistas.

4.4. Ejemplo Práctico con Múltiples Tablas

Supongamos una base de datos de ventas con las tablas:

- `clientes` (id, nombre, ciudad)
- `productos` (id, nombre, precio)
- `pedidos` (id, cliente$_id, fecha$)
- `detalle_pedido` (pedido$_id, producto_id, cantidad$)

Queremos obtener una lista de pedidos con nombre del cliente, fecha, productos y cantidad:

```sql
SELECT c.nombre AS cliente, p.fecha, pr.nombre AS producto, dp.
   cantidad
FROM pedidos p
JOIN clientes c ON p.cliente_id = c.id
JOIN detalle_pedido dp ON p.id = dp.pedido_id
JOIN productos pr ON dp.producto_id = pr.id;
```

Este es un JOIN de cuatro tablas. El orden de los JOINs puede afectar el rendimiento, pero el optimizador suele reordenarlos.

Capítulo 5

Subconsultas

Una subconsulta es una consulta anidada dentro de otra. Pueden usarse en SELECT, FROM, WHERE, etc. Son herramientas poderosas pero a veces pueden reescribirse como JOINs para mejorar rendimiento.

5.1. Subconsultas Escalares

Devuelven un solo valor (una fila y una columna). Se usan en expresiones.

```
SELECT nombre, (SELECT AVG(precio) FROM productos) AS
   precio_medio
FROM productos;
```

Deben asegurar que devuelven una sola fila; si devuelven varias, se produce error.

5.2. Subconsultas de Fila Única

Devuelven una fila con varias columnas. Se pueden comparar con operadores como `=`, `>`, `<` si se espera una sola fila.

```
SELECT * FROM productos
WHERE (categoria, precio) = (SELECT categoria, MAX(precio) FROM
   productos GROUP BY categoria LIMIT 1);
```

(En la práctica, es más seguro usar EXISTS o IN.)

5.3. Subconsultas de Múltiples Filas

Devuelven varias filas. Se usan con `IN`, `ANY`, `ALL`, `EXISTS`.

- **IN**: Verifica si el valor está en el conjunto.
- **ANY / SOME**: Compara con algún valor del conjunto (ej. ¿ANY significa mayor que al menos uno).
- **ALL**: Compara con todos los valores (ej. ¿ALL significa mayor que todos).
- **EXISTS**: Verifica si la subconsulta devuelve alguna fila (es más eficiente que IN para conjuntos grandes porque puede parar al encontrar la primera).

```
SELECT nombre FROM clientes
WHERE id IN (SELECT cliente_id FROM pedidos WHERE fecha >= '
   2025-01-01');

SELECT nombre FROM clientes c
WHERE EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id
   AND p.fecha >= '2025-01-01');
```

`EXISTS` suele ser más rápido cuando la subconsulta es correlacionada.

5.4. Subconsultas Correlacionadas

Hacen referencia a columnas de la consulta exterior. Se evalúan para cada fila de la consulta externa.

```
SELECT p.nombre, p.precio
FROM productos p
WHERE p.precio > (SELECT AVG(precio) FROM productos WHERE
   categoria = p.categoria);
```

Aquí, la subconsulta calcula el precio medio de la categoría del producto actual. Pueden ser lentas si la tabla externa es grande.

5.5. Subconsultas en la cláusula FROM

Se puede usar una subconsulta como una tabla derivada (vista temporal). Requiere un alias.

```
SELECT categoria, AVG(precio) AS precio_medio
FROM (SELECT * FROM productos WHERE activo = true) AS
   productos_activos
GROUP BY categoria;
```

Capítulo 6

Common Table Expressions (CTEs)

Las CTEs (WITH queries) permiten definir consultas auxiliares que se comportan como tablas temporales dentro de una consulta principal. Mejoran la legibilidad y permiten consultas recursivas.

6.1. Sintaxis Básica

```
WITH cte_nombre (columnas_opcional) AS (
    consulta
)
SELECT ...
FROM cte_nombre ...
```

6.1.1. Ejemplo

Obtener el total de ventas por cliente y luego filtrar los que superan 1000:

```
WITH ventas_cliente AS (
    SELECT c.id, c.nombre, SUM(dp.cantidad * pr.precio) AS total
    FROM clientes c
    LEFT JOIN pedidos p ON c.id = p.cliente_id
    LEFT JOIN detalle_pedido dp ON p.id = dp.pedido_id
    LEFT JOIN productos pr ON dp.producto_id = pr.id
    GROUP BY c.id, c.nombre
)
SELECT * FROM ventas_cliente WHERE total > 1000;
```

6.1.2. CTEs Múltiples

Se pueden definir varias CTEs separadas por comas.

```
WITH
    ventas AS (SELECT ...),
    clientes_vip AS (SELECT ...)
SELECT ...
```

6.1.3. CTEs Recursivas

Permiten consultar estructuras jerárquicas (ej. organigrama). La sintaxis incluye una parte no recursiva (anchor) y una parte recursiva unida con UNION ALL.

```
WITH RECURSIVE organigrama AS (
    -- Anchor: el jefe m ximo
    SELECT id, nombre, jefe_id, 1 AS nivel
    FROM empleados
    WHERE jefe_id IS NULL
    UNION ALL
    -- Recursivo: empleados que tienen jefe
    SELECT e.id, e.nombre, e.jefe_id, o.nivel + 1
    FROM empleados e
    JOIN organigrama o ON e.jefe_id = o.id
)
SELECT * FROM organigrama;
```

Es una pregunta común en entrevistas avanzadas.

6.2. CTEs vs. Subconsultas vs. Tablas Temporales

- Las CTEs son solo azúcar sintáctico; el optimizador puede tratarlas como subconsultas en línea o materializarlas según el motor (en PostgreSQL, se materializan por defecto a menos que se use `NOT MATERIALIZED`).
- Son más legibles y permiten recursión.
- Para reutilización múltiple en la misma consulta, una CTE evita repetir código.

Capítulo 7

Buenas Prácticas y Optimización de Consultas SQL

Escribir SQL eficiente no solo es cuestión de sintaxis correcta, sino de entender cómo el motor ejecutará la consulta. Aquí hay reglas y patrones que suelen preguntar en entrevistas.

7.1. Preferir EXISTS a IN cuando la subconsulta es grande

```
-- Menos eficiente si la subconsulta devuelve muchas filas
SELECT * FROM clientes WHERE id IN (SELECT cliente_id FROM
   pedidos);

-- M s eficiente (puede parar al encontrar el primero)
SELECT * FROM clientes c WHERE EXISTS (SELECT 1 FROM pedidos p
   WHERE p.cliente_id = c.id);
```

7.2. Evitar Funciones en Columnas Indexadas

```
-- No usar   ndice  en fecha
SELECT * FROM pedidos WHERE EXTRACT(YEAR FROM fecha) = 2025;

-- Mejor (si hay  ndice  en fecha)
SELECT * FROM pedidos WHERE fecha >= '2025-01-01' AND fecha < '
   2026-01-01';
```

7.3. Usar UNION ALL en lugar de UNION si no se necesitan duplicados

UNION elimina duplicados (requiere ordenar y comparar), mientras que UNION ALL simplemente concatena.

7.4. SELECT * Es Peligroso

Siempre especificar las columnas necesarias. SELECT * puede traer más datos de los necesarios, aumentar E/S y red, y romper la aplicación si cambia el esquema.

7.5. Cuidado con las Conversiones Implícitas

Si comparas un texto con un número, el motor puede convertir toda la columna, impidiendo el uso de índices.

```
-- Si id es texto, pero lo comparas con n mero , puede haber
   conversi n
SELECT * FROM tabla WHERE id = 123; -- mejor usar '123'
```

7.6. Índices y JOINs

Asegurar que las columnas de JOIN estén indexadas. Para JOINs de muchos a muchos, indexar ambas columnas en la tabla intermedia.

7.7. Analizar el Plan con EXPLAIN

Siempre que una consulta sea lenta, usar EXPLAIN (ANALYZE) para ver dónde está el cuello de botella.

7.8. Escritura de Consultas Legibles

Usar indentación, alias claros, y mayúsculas para palabras clave. No afecta el rendimiento pero facilita el mantenimiento.

Capítulo 8

Consideraciones de Hardware y Motor

8.1. Buffer Pool y Caché

El tamaño del buffer pool (por ejemplo, `innodb_buffer_pool_size` en MySQL, `shared_buffers` en PostgreSQL) es crítico. Si los datos caben en memoria, las consultas son mucho más rápidas.

8.2. Disco: SSD vs HDD

En producción, siempre SSD. Las lecturas secuenciales son más rápidas, pero los índices requieren lecturas aleatorias. SSD mejora drásticamente las lecturas aleatorias.

8.3. Parallel Query

Algunos motores pueden usar múltiples CPUs para una consulta (paralelismo). Esto puede acelerar agregaciones y escaneos.

8.4. Configuración del Motor

Parámetros como el tamaño de memoria para hash joins, el costo de E/S, etc., pueden ajustarse. En entrevistas, suelen preguntar sobre parámetros genéricos.

Capítulo 9

Breve Comparación entre Motores (para JOINs y DML)

Aunque nos centramos en SQL estándar, cada motor tiene peculiaridades:

- **PostgreSQL**: Cumple con el estándar, soporta CTEs recursivas, FULL JOIN, y tiene un optimizador muy maduro.
- **MySQL**: No soporta FULL JOIN (se simula con UNION). Los JOINs pueden ser lentos si no se usan índices.
- **SQL Server**: Ofrece sugerencias de JOIN (LOOP, HASH, MERGE) y soporta CTEs recursivas.
- **Oracle**: Sintaxis antigua de JOIN (+) para outer joins, pero se recomienda usar ANSI.

En entrevistas, es importante conocer las diferencias si el puesto requiere un motor específico.

Capítulo 10

Joins en el Mundo NoSQL (Una Breve Mirada)

Aunque esta sesión se centra en SQL, es útil saber cómo se abordan las relaciones en bases de datos NoSQL, ya que en proyectos modernos pueden convivir ambos paradigmas.

- **MongoDB (documental):** No soporta joins nativos entre colecciones, pero desde la versión 3.2 incluye el operador `$lookup` en el pipeline de agregación, que realiza un left outer join con otra colección. Sin embargo, su uso es limitado y se recomienda diseñar documentos embebidos para evitar joins.
- **Cassandra (columna ancha):** No soporta joins. El modelado se basa en desnormalización y tablas diseñadas específicamente para las consultas.
- **Redis (clave-valor):** No tiene joins; las relaciones se manejan a nivel de aplicación o mediante estructuras como sets y sorted sets.
- **Neo4j (grafos):** Las relaciones son nativas; en lugar de joins, se navegan los nodos y relaciones con patrones de camino.

En resumen, en NoSQL se sacrifica la capacidad de join en favor de escalabilidad y rendimiento, por lo que el diseño de datos es fundamental.

Capítulo 11

Ejercicios Resueltos

A continuación se presentan ejercicios que integran los conceptos vistos. Se resuelven paso a paso.

11.1. Ejercicio 1: Consultas Básicas con DML

Enunciado: Dada la tabla `empleados` con columnas `id`, *nombre*, *salario*, *departamento_id*. Insertar tres empleados, actualizar el salario de uno, y luego eliminar los empleados de un departamento específico.

Solución:

```
-- Insertar
INSERT INTO empleados (nombre, salario, departamento_id) VALUES
('Juan Perez', 3000, 1),
('Ana Garcia', 3500, 2),
('Luis Fernandez', 3200, 1);

-- Actualizar salario de Juan
UPDATE empleados SET salario = 3300 WHERE nombre = 'Juan Perez';

-- Eliminar empleados del departamento 2
DELETE FROM empleados WHERE departamento_id = 2;
```

11.2. Ejercicio 2: INNER JOIN y LEFT JOIN

Enunciado: Usar las tablas `clientes`, `pedidos` y `detalle_pedido` del ejemplo anterior. Obtener:

1. Todos los pedidos con el nombre del cliente y la fecha.
2. Todos los clientes, incluso si no tienen pedidos, mostrando el número de pedidos (0 si no tiene).
3. El total gastado por cada cliente (suma de cantidad * precio de cada producto en sus pedidos).

Solución:

```sql
-- 1. Pedidos con cliente
SELECT p.id, c.nombre, p.fecha
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.id;

-- 2. Todos los clientes con n mero de pedidos
SELECT c.id, c.nombre, COUNT(p.id) AS num_pedidos
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nombre;

-- 3. Total gastado por cliente
SELECT c.id, c.nombre, COALESCE(SUM(dp.cantidad * pr.precio), 0)
   AS total_gastado
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
LEFT JOIN detalle_pedido dp ON p.id = dp.pedido_id
LEFT JOIN productos pr ON dp.producto_id = pr.id
GROUP BY c.id, c.nombre;
```

Explicación: Usamos LEFT JOIN para incluir clientes sin pedidos, y COALESCE para mostrar 0 en lugar de NULL.

11.3. Ejercicio 3: Subconsultas

Enunciado: Encontrar los productos cuyo precio es superior al precio medio de su categoría. Usar subconsulta correlacionada.

Solución:

```sql
SELECT p.nombre, p.precio, p.categoria
FROM productos p
WHERE p.precio > (SELECT AVG(precio) FROM productos WHERE
   categoria = p.categoria);
```

11.4. Ejercicio 4: Uso de CTE

Enunciado: Obtener los empleados que ganan más que el salario promedio de su departamento, mostrando el nombre del empleado, su salario y el promedio del departamento.

Solución:

```sql
WITH promedio_depto AS (
    SELECT departamento_id, AVG(salario) AS salario_prom
    FROM empleados
    GROUP BY departamento_id
)
SELECT e.nombre, e.salario, pd.salario_prom
FROM empleados e
JOIN promedio_depto pd ON e.departamento_id = pd.departamento_id
```

```
WHERE e.salario > pd.salario_prom;
```

11.5. Ejercicio 5: Optimización y EXPLAIN

Enunciado: Dada una consulta lenta, ¿cómo diagnosticarías y optimizarías? Supongamos: `SELECT * FROM pedidos WHERE EXTRACT(YEAR FROM fecha) = 2025;`

Solución:

1. Usar EXPLAIN ANALYZE para ver el plan.
2. Probablemente hará un seq scan porque la función impide usar índice en fecha.
3. Reescribir: `SELECT * FROM pedidos WHERE fecha >= '2025-01-01' AND fecha <'2026-01-01`
4. Crear índice en `fecha` si no existe.
5. Verificar con EXPLAIN que ahora use index scan.

11.6. Ejercicio 6: Recursividad con CTE

Enunciado: Dada una tabla `empleados` con `id`, *nombre*, *jefe_id*, obtener el organigrama completo con niveles.

Solución:

```
WITH RECURSIVE organigrama AS (
    SELECT id, nombre, jefe_id, 1 AS nivel
    FROM empleados
    WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, o.nivel + 1
    FROM empleados e
    JOIN organigrama o ON e.jefe_id = o.id
)
SELECT * FROM organigrama ORDER BY nivel, nombre;
```

Capítulo 12

Glosario de Términos

DML Data Manipulation Language: sublenguaje de SQL para manipular datos (SELECT, INSERT, UPDATE, DELETE).

JOIN Operación que combina filas de dos o más tablas basándose en una condición relacionada.

INNER JOIN Devuelve solo las filas que tienen correspondencia en ambas tablas.

LEFT JOIN Devuelve todas las filas de la tabla izquierda, y las coincidentes de la derecha.

FULL OUTER JOIN Devuelve todas las filas de ambas tablas, rellenando con NULL donde no hay coincidencia.

Subconsulta Consulta anidada dentro de otra consulta.

Subconsulta correlacionada Subconsulta que hace referencia a columnas de la consulta exterior.

CTE Common Table Expression (WITH clause): consulta auxiliar que puede ser referenciada múltiples veces.

Recursive CTE CTE que se llama a sí misma, útil para jerarquías.

EXPLAIN Comando para mostrar el plan de ejecución de una consulta.

Plan de ejecución Secuencia de operaciones que el motor realizará para ejecutar una consulta.

Buffer Pool Área de memoria donde se cachean páginas de datos e índices.

Index Scan Acceso a tabla mediante un índice.

Seq Scan Escaneo secuencial de toda la tabla.

Referencias

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*. 7th ed. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts*. 7th ed. McGraw-Hill.
- Documentación oficial de PostgreSQL: `https://www.postgresql.org/docs/`
- Documentación de MySQL: `https://dev.mysql.com/doc/`
- "Use The Index, Luke" – Guía de optimización de SQL: `https://use-the-index-luke.com/`